Registro de Desarrollo de API en NestJS

Consulta / Prompt Principal	**Objetivo de la Consulta**	**Información / Palabras Clave Obtenidas**	**Transición a la Siguiente Fase**
"Necesito hacer un ejercicio... crear un nuevo proyecto, conectar con Postman y lograr tener usuarios temporales"	Iniciar el proyecto de NestJS desde cero y entender cómo guardar datos sin usar una base de datos.	CLI de NestJS (nest new, nest g resource), setTimeout, memoria temporal.	Una vez con los pasos claros, pasamos a ver dónde íbamos a instalar el proyecto en Visual Studio Code.
"Tengo creada esta carpeta vacía (H-Ejercicio) para no perderme..." + capturas de la terminal.	Configurar bien la carpeta del proyecto en VS Code y crear los archivos base de los usuarios sin liarla con las rutas.	nest new ., Modo restringido (Trust Workspace), rutas de la terminal.	Con la estructura creada, metimos el código del servicio temporal, pero empezaron a saltar errores.
"Me sigue tirando error TS2345 y TS2339" (mostrando logs de la consola).	Arreglar los errores de compilación porque el controlador autogenerado no coincidía con el código que hicimos.	Errores de TypeScript, limpieza de métodos, guardar cambios (Ctrl+S).	Ya con la terminal "en verde" y compilando bien, abrimos Postman para empezar a probar.
"Revisión de la respuesta en la interfaz de Postman tras ejecutar el POST y validación del GET."	Probar en Postman que el POST creara al usuario y confirmar que el temporizador de 60s lo borraba en el GET.	Status 201 Created, JSON Body, pruebas de GET y POST.	Como lo básico ya funcionaba, tocaba agregar las cosas extra que me iban a evaluar en el código.
"Necesito que HEAD y OPTIONS estén explicados en el código la parte para que funcionen".	Implementar estos dos métodos y dejarles comentarios técnicos para que el profe vea que entiendo cómo funcionan.	@Head(), @Options(), @Res(), solución al error de importación estricta (type { Response }).	Aprovechando que estábamos comentando el código, decidí meter de una vez todo el CRUD.
"Lo de PUT, PATCH y DELETE también se pueden encontrar en el código?"	Dejar el proyecto súper completo agregando los métodos que faltaban para actualizar y borrar manualmente, con	CRUD completo, @Put(), @Delete(), comentarios JSDoc.	Al ver cómo funcionaba la eliminación manual, me surgió la duda de qué estábamos usando para guardar la info.

	sus explicaciones.		
"Oye una pregunta, ¿usaste arrays?"	Entender la estructura de datos que usamos en el código para simular la base de datos en la memoria RAM.	Arreglos (any[]), métodos de JavaScript (push, filter, findIndex, splice).	Al entender los arrays, noté que se podía hacer el código más estricto con TypeScript.
"¿Por qué un ejemplo que vi tiene un user.interface.ts y el mío no? Ya me dio error al modificar el user.service.ts"	Mejorar la calidad del proyecto usando interfaces para que los datos tengan tipos estrictos (name, email, age).	Tipado estricto (interface User), convertir strings a números con el signo +id.	Ya con el tipado estricto arreglado y sin errores, solo faltaba pulir lo que iba a entregar.
"Vale ahora me recuerdas lo del user.controller.ts... y vale quita el patch"	Darle una repasada final a cómo funciona el controlador para la revisión y borrar métodos que no quería incluir en la entrega.	Repaso de decoradores (@Body, @Param), código final limpio.	Fin: Proyecto listo, funcional, bien comentado y tipado para subir a GitHub.